Threading-debugging exercises

We have six bugs, each demonstrating one of the techniques covered in the lecture. All examples use OpenMP for the parallel constructs.

Build defaults: `gcc -O1 -g3 -fno-omit-frame-pointer -fopenmp`; add `-fsanitize=thread` whenever the lecture says so.

The bugs are presented in roughly the order the lecture covers them, and each one is self-contained — read the header comment of the relevant `.c` file before running anything; the comments are part of the exercise.

A word about your hardware

Several of these exercises rely on the OS actually scheduling threads on different CPUs. On a machine with only **one** logical CPU available — a constrained VM, a busy laptop, a container with a CPU quota — OpenMP threads time-slice rather than run in parallel, and races may **fail to manifest** on plain runs. So, despiete the fact that the bug is in the source, the schedule decides whether you observe it.

When in doubt, run under TSan. TSan reasons about happens-before rather than wall-clock interleaving and finds the race regardless of how many CPUs you have.

Exercise 1 — the canonical race

File: `0_race_counter.c`
Slides: 3 (the race), 4 (TSan)

A shared counter incremented by two OpenMP threads with no synchronisation.

1. Build and run without sanitizers, at `-O0` and at `-O2`:

   ```
   gcc -O0 -g3 -fopenmp -o race 0_race_counter.c   ;  ./race
   gcc -O2 -g3 -fopenmp -o race 0_race_counter.c   ;  ./race
   ```

   The final value is wrong by varying amounts, and `-O2` typically makes it worse. (If your machine has only one CPU the race may not fire on plain runs — see the note above.)

2. Rebuild under ThreadSanitizer and read the report:

   ```
   gcc -O1 -g3 -fopenmp -fsanitize=thread -o race 0_race_counter.c
   ./race
   ```

   The diagnostic names the function as `main._omp_fn.0` (libgomp's outlined parallel region), the line as the one with `counter++`, and gives two stack traces — one through `GOMP_parallel`, one through the worker entry of libgomp.

3. Fix the bug in one of the three ways the header comment lists (`#pragma omp atomic`, `#pragma omp critical`, or `reduction`); re-run TSan, verify you have now a clean run.

Exercise 2 — deadlock

File: `1_deadlock.c`
Slides: 2 (gdb with threads), 5 (deadlock)

Two OpenMP sections, two locks, opposite lock orders.

1. Build and run. The program hangs.
2. From a second terminal, attach with gdb and inspect:

   ```
   ./deadlock &
   gdb -p $!
   (gdb) thread apply all bt
   ```

   Two worker threads, both blocked at `omp_set_lock` → `gomp_mutex_lock_slow` → `__lll_lock_wait`, called from `main._omp_fn.0`. Identify which section holds which lock and which is waiting on which; convince yourself the graph has a cycle.
3. Build under TSan (the deadlock-inducing `usleep`s can stay or go — TSan still detects the lock-order inversion):

   ```
   gcc -O1 -g3 -fopenmp -fsanitize=thread -o deadlock 1_deadlock.c
   ```

   Notice that TSan finds the lock-order problem from a single run, before the production hang.

Exercise 3 — barriers

File: `2_missing_barrier.c`
Slides: 6

Two phases inside a single `parallel` region with no explicit barrier between them.

1. Build and run with several thread counts:

   ```
   gcc -O2 -g3 -fopenmp -o nobarrier 2_missing_barrier.c
   OMP_NUM_THREADS=4 ./nobarrier
   OMP_NUM_THREADS=8 ./nobarrier
   ```

   Inspect the printed "neighbour" values; note the ones that read the sentinel `-1` (the neighbour had not yet written).
2. Add `#pragma omp barrier` between phase 1 and phase 2; re-run; verify stability.
3. Run under TSan with and without the fix.

Exercise 4 — reduction

File: `3_reduction_wrong.c`
Slides: 6

A loop accumulating into a scalar, parallelised without `reduction(+:sum)`.

1. Build and run:

   ```
   gcc -O2 -g3 -fopenmp -o reduce 3_reduction_wrong.c
   OMP_NUM_THREADS=4 ./reduce
   ```

   `good_sum` is correct; `bad_sum` is wrong, by an amount that varies between runs.

2. Run under TSan; confirm the race is on `sum`.

3. Notice that adding `default(none)` to the `#pragma omp parallel for` clause forces you to declare every variable as shared, private, or reductioned — which would have caught the bug at compile time.

Exercise 5 — the heisenbug

File: `4_heisenbug_printf.c`
Slides: 7

The race of exercise 1, with a "debug" `printf` inside the loop that fixes the race by accident.

1. Build without the printf and confirm the race:

   ```
   gcc -O2 -g3 -fopenmp -o heisen 4_heisenbug_printf.c
   ./heisen
   ```

2. Build with the printf and observe that the race appears to be fixed:

   ```
   gcc -O2 -g3 -fopenmp -DUSE_PRINTF -o heisen 4_heisenbug_printf.c
   ./heisen > /dev/null
   ```

3. Build with the printf AND under TSan, and observe that TSan reports the race anyway:

   ```
   gcc -O1 -g3 -fopenmp -DUSE_PRINTF -fsanitize=thread -o heisen 4_heisenbug_printf.c
   ./heisen > /dev/null
   ```

   The race is a property of the source. Whether you observe it in a given run is a property of the schedule.

Exercise 6 — attaching when you would not otherwise

File: `5_spin_wait_attach.c`
Slides: 8 (spin-wait trick)

A program that prints its PID and spins on a flag before entering its OpenMP parallel region.

1. Build and run in one terminal:

   ```
   gcc -O0 -g3 -fopenmp -o spin 5_spin_wait_attach.c
   ./spin
   ```

Note the PID printed.

2. In a second terminal, attach:

```
gdb -p <PID>
(gdb) print spin_flag
(gdb) set spin_flag = 1
(gdb) continue
```

   The master thread escapes the loop and enters the parallel region. At that point `info threads` (D2) shows the OpenMP worker pool.

3. Try setting a breakpoint inside `do_work` before continuing, so you stop inside the parallel region with full debugger control.

4. Wrap `wait_for_debugger()` in `#ifdef SPIN_FOR_DEBUGGER`; rebuild without that define; confirm the wait disappears. This is the form to keep in a real codebase.